

Report

Some figures in this report come from additional exploratory notebooks used during development and not included in the submitted repository.

Task 1. Named Entity Recognition — Mountain Names

Results

All models were evaluated on the same held-out test set.

Model	Scheme	Precision	Recall	F1
DeBERTa-v3-base	binary	0.833	0.853	0.843
DeBERTa-v3-base	BIO (<i>additional notebook</i>)	0.829	0.854	0.841
bert-base-cased	binary	0.826	0.854	0.840
bert-base-cased	BIO	0.824	0.854	0.839
distilbert-base-cased	binary	0.801	0.830	0.815

DeBERTa edges out BERT in both schemes, though not by much, and the labeling scheme itself barely moves F1 within a given architecture. DistilBERT gives up about 2.4 points of F1 but trains in roughly half the time, which seems like a fair trade if speed matters more than the last bit of accuracy. Recall beats precision across the board — that tracks with the label noise mentioned below, since the model has learned to tag some generic nouns that aren't actually mountains.

The F1 gaps are small enough that they don't tell the full story, so a handful of hand-picked examples were checked too (from an additional model-checking notebook, not in the submitted repo). A few things stood out:

- On "a data scientist in Kyiv", bert-base-cased (both schemes) and DeBERTa-BIO tag Kyiv as a mountain. DeBERTa-binary and DistilBERT don't. Every model correctly ignored "the Amazon river" in a separate sentence, for what it's worth.
- "the summit" with no mountain named gets tagged anyway — again, this comes straight from the label noise in the training data.
- In a longer paragraph about the Himalayas, Annapurna got missed while Dhaulagiri right next to it was picked up with the highest confidence in the whole text. A recall of 0.85 means this kind of thing happens.
- Because the data was tokenized by whitespace, "Everest." and "Everest" are technically different tokens, and the model is noticeably less confident on the one with the period (0.964 vs 0.980, DeBERTa-binary).
- Confidence isn't always trustworthy either — bert-base-cased was more confident about its wrong answer (Kyiv, 0.830) than its right one (Kilimanjaro, 0.786). A fixed confidence cutoff wouldn't have caught that.

DeBERTa-v3-base on the binary scheme was the final pick — best F1, one of only two models that reject Kyiv, and it held up best on the edge cases above. The cost is a heavier tokenizer dependency (SentencePiece) and one fp32-loading fix, both already handled in train.py.

Proposed improvements

1. About 6.61% of the mountain-tagged tokens are lowercase generic nouns (summit, peak, ridge...), and this noise shows up directly in the model's mistakes — it still tags a bare "summit" as a mountain even with no specific peak mentioned. Filtering or manually reviewing these cases would probably be the single highest-value fix here.
2. Kyiv gets tagged as a mountain by two of the four models. Adding cities and rivers as explicit negative examples during training should help — right now "proper noun in a geographic sentence" seems to be enough on its own to trigger a false positive.
3. The whitespace tokenization causes a small but measurable issue: "Everest." gets a noticeably different (lower) confidence than "Everest", just because of the attached period. Switching to a real tokenizer would remove this artifact.

4. bert-base-cased's confidence scores aren't well calibrated — in one case a false positive scored higher than a true positive. Temperature scaling or a similar method could make confidence thresholding more trustworthy.
5. The task suggested using an LLM for dataset generation. That wasn't needed for volume here, but it could still help with rare names specifically — Annapurna, for instance, is one the model currently misses.
6. DeBERTa already beats BERT by a small margin. A larger model, or an ensemble of what's already trained, could be worth trying if there's compute budget for it.

Task 2. Computer Vision — Sentinel-2 Image Matching

Results

Matcher	Best channel	mma_3px	Mean correct matches	Winter pair
roma	B03	0.95–0.99	~2048	~1977 (no degradation)
aliked-lightglue	B03	0.85	601.8	315
eloftr	—	lower	1093	collapses
xfeat-lightglue	B02	lower	—	collapses
sift-lightglue	—	lower	—	collapses
SIFT	—	lowest	—	collapses

RoMa is the standout here — winter performance basically matches its performance everywhere else, which none of the other five matchers come close to. aliked-lightglue is the best of the rest and is what the submitted code actually defaults to, since it's much lighter and gives explicit keypoints.

Fine-tuning experiment

LightGlue's matcher was fine-tuned with ALIKED frozen as the detector — LightGlue is the part meant to be adapted, and fine-tuning a detector from scratch is a bigger, riskier job (it would need something like SuperPoint's self-supervised scheme, and could easily make the detector worse rather than better). Training used a custom loop on top of cvg/LightGlue, with ground truth coming from the same synthetic homographies used for evaluation, split by geographic block to avoid leakage between train and validation.

The official LightGlue training pipeline, glue-factory, wasn't a realistic option — it wants the Oxford-Paris dataset (~450 GB) and MegaDepth (~420 GB), plus two 24 GB GPUs by default. Worth noting too: an open issue on that repo (cvg/glue-factory#113) describes someone plugging in their own dataset and getting zero matches out, unresolved — so even the official path isn't obviously smooth for custom data. A smaller custom loop seemed like the more realistic bet given the time available.

Three variants were tried (in a separate notebook, not part of the submitted train.py), and none got close to the pretrained baseline:

Variant	Loss formulation	Mean correct matches	Precision
pretrained	—	601.8	0.85
v2	single-layer loss (final layer supervised, rest of the network updated through it)	1.5	~0.10
v3	multi-layer, uniform average across all 9 layers	0.8	0.009

Both fine-tuned versions collapsed almost completely. The likely explanation is that LightGlue's real training recipe weights the per-layer loss unevenly — early layers are naturally less confident than later ones, so averaging them uniformly (v3) probably let noisy early-layer gradients drag down the layers that actually matter. That's a specific

enough failure mode that it seemed better to stop and report it honestly than keep guessing at loss weights. Pretrained weights are what ships in the final pipeline.

Demo results

Results on B03:

Case	ALIKED + LightGlue	RoMa
Easy — summer ↔ summer, 3 years apart	1068 matches, 99.2% inliers	2048 matches, 100% inliers
Medium — summer ↔ autumn, 3 months apart	1026 matches, 99.8% inliers	2048 matches, 100% inliers
Hard — winter ↔ summer	784 matches, 80.6% inliers	2048 matches, 100% inliers

RoMa's inlier rate doesn't budge across any of the three cases — it just doesn't seem to notice the season changing. The sparse matcher does, dropping to 80.6% on the hard case, though it's still usable there.

Proposed improvements

1. Two ways to actually fix the fine-tuning: either commit to [glue-factory](#) properly, hardware requirements and all, or reimplement LightGlue's actual per-layer loss weighting from the paper — neither was realistic in this project's timeframe. SuperPoint might be the easier path overall, since its self-supervised training doesn't have this per-layer balancing problem to begin with.
2. RoMa was only ever tested on B03, inherited from what worked best for aliked-lightglue. Given that xfeat-lightglue does better on B02 instead, there's a real chance RoMa's actual best channel hasn't been found yet.
3. Every number in this report comes from a synthetic homography, because the patches are pixel-aligned and there's no ground truth for genuinely new pairs. Sentinel-2 scenes are georeferenced, though, so two independently downloaded scenes of the same place could provide real correspondence — that's the setting this is actually meant for, and it hasn't been tested yet.

4. Both the custom NDSI filter and s2cloudless miss thin, faint clouds — confirmed by eye on a couple of winter patches that technically passed a 0.1% cloud threshold. A cirrus-band (B10) check or some texture-based test might catch what brightness alone can't.
5. RoMa hits 100% inliers on all three demo cases, so the demo can't really show where its limits are. Something harder — unregistered scenes, bigger synthetic warps, partial overlaps — would probably be more informative.
6. The pipeline only handles 512×512 patches right now, and a full Sentinel-2 tile is 10,980×10,980. Getting it to work on whole scenes would mean sliding-window matching with overlap and stitching the results back together — also just the right way to not lose resolution, per the task description.
7. Every matcher here only ever sees one grayscale channel at a time, even though the channels clearly carry different information — B03 wins overall, B02 is better for xfeat-lightglue, B11 has its own SWIR signal. Stacking or fusing channels could combine some of that, especially for snow, where the visible bands lose most of their contrast.
8. T36UXA got dropped from the main dataset early on because its available scenes don't have real seasonal contrast. With the right pair, though, it could still work as a held-out region.